

Programming Language Principles, Design, and Implementation

Lecture 6 - Small-Step Operational Semantics

Sean Moss

University of Birmingham

Where are we?

- ▶ **Part 1: Principles of programming**
 - ▶ Functional languages
 - ▶ **Operational semantics**
 - ▶ Type systems
 - ▶ Abstract datatypes
- ▶ Part 2: Compilers

Today

Small-step operational semantics

- ▶ Operational semantics as transition systems
- ▶ Evaluation contexts
- ▶ Normal forms

Recap: Operational semantics

Structural operational semantics

- ▶ Specifies the behaviour of programs by describing how they execute on abstract machines that perform steps of computations
- ▶ Provides a link to implementations
- ▶ Rules are specified in a syntax directed way

Several kinds:

- ▶ **Big-step** (last lecture)

Specifies how programs execute in 1 step

Closely models a recursive interpreter

Sometimes leads to smaller proofs (less rules)

- ▶ **Small-step** (covered in this lecture)

Specifies how programs execute 1 step at a time

Easier to model and reason about complex features such as concurrency

Recap: Call-by-value vs. call-by name big-step semantics

Call-by-value

$$\overline{\lambda x.M \Rightarrow_v \lambda x.M} \text{ VAL}$$

$$\frac{M \Rightarrow_v \lambda x.P \quad N \Rightarrow_v Q \quad P[x \backslash Q] \Rightarrow_v V}{M \ N \Rightarrow_v V} \text{ APP}$$

Call-by-name

$$\overline{\lambda x.M \Rightarrow_n \lambda x.M} \text{ VAL}$$

$$\frac{M \Rightarrow_n \lambda x.P \quad P[x \backslash N] \Rightarrow_n V}{M \ N \Rightarrow_n V} \text{ APP}$$

You can interpret these rules as allowing free variables in terms, but there is a high chance of evaluation getting stuck.

Small-step operational semantics

Big-step

- ▶ terms are evaluated in “1 big step”
- ▶ specifies how to implement an interpreter
- ▶ abstract: sometimes lead to easier proofs
- ▶ hard to handle concurrency
- ▶ hides non-termination (lack of proof)

Small-step semantics overcome these shortcomings

Specified by a **transition system**

- ▶ **states** are λ -terms
- ▶ **transitions** are provided by inference rules

Small-step call-by-value operational semantics

Values (λ -abstractions of the form $\lambda x.M$) are denoted by V

β rule:

$$\overline{(\lambda x.M) V \longrightarrow_v M[x \setminus V]}^{\beta}$$

Context rules:

$$\frac{M \longrightarrow_v P}{M N \longrightarrow_v P N} \text{APP}_1$$

$$\frac{M \longrightarrow_v P}{V M \longrightarrow_v V P} \text{APP}_2$$

Let $\longrightarrow_v^*$ be the reflexive and transitive closure of $\longrightarrow_v$

$$M \longrightarrow_v^* N \iff (M = N \vee \exists P. M \longrightarrow_v P \wedge P \longrightarrow_v^* N)$$

Small-step call-by-value operational semantics

Booleans:

$$\begin{array}{llll} \text{T} & = & \lambda x. \lambda y. x & \text{and} & = & \lambda a. \lambda b. a \ b \ \text{F} \\ \text{F} & = & \lambda x. \lambda y. y & \text{or} & = & \lambda a. \lambda b. a \ \text{T} \ b \\ & & & \text{not} & = & \lambda a. a \ \text{F} \ \text{T} \end{array}$$

What does $(\text{or } \text{T} \ \text{F})$ compute to according to this semantics?

$$\frac{\overline{\text{or } \text{T} \longrightarrow_v \lambda b. \text{T} \ \text{T} \ b} \beta}{\text{or } \text{T} \ \text{F} \longrightarrow_v (\lambda b. \text{T} \ \text{T} \ b) \ \text{F}} \text{APP1}$$

$$\frac{}{(\lambda b. \text{T} \ \text{T} \ b) \ \text{F} \longrightarrow_v \text{T} \ \text{T} \ \text{F}} \beta$$

$$\frac{\overline{\text{T} \ \text{T} \longrightarrow_v \lambda y. \text{T}} \beta}{\text{T} \ \text{T} \ \text{F} \longrightarrow_v (\lambda y. \text{T}) \ \text{F}} \text{APP1}$$

$$\frac{}{(\lambda y. \text{T}) \ \text{F} \longrightarrow_v \text{T}} \beta$$

Therefore: $\text{or } \text{T} \ \text{F} \longrightarrow_v^* \text{T}$

Small-step call-by-value operational semantics

Let $\Delta = \lambda x.xx$

What does $(\Delta\Delta)$ compute to according to this semantics?

$$\overline{\Delta\Delta \longrightarrow_v \Delta\Delta}^\beta$$

We are back to the same term

There is no value V such that $\Delta\Delta \longrightarrow_v^* V$!

Small-step call-by-name operational semantics

Arguments of functions are only evaluated when used

β rule:

$$\frac{}{(\lambda x.M) N \longrightarrow_n M[x \backslash N]} \beta$$

Context rule:

$$\frac{M \longrightarrow_n P}{M N \longrightarrow_n P N} \text{APP}$$

Let $\longrightarrow_n^*$ be the reflexive and transitive closure of $\longrightarrow_n$

$$M \longrightarrow_n^* N \iff (M = N \vee \exists P. M \longrightarrow_n P \wedge P \longrightarrow_n^* N)$$

Small-step call-by-name operational semantics

Let $\Delta = \lambda x.xx$

What does $(\lambda x.\lambda y.y)(\Delta\Delta)$ compute to according to this semantics?

$$\overline{(\lambda x.\lambda y.y)(\Delta\Delta)} \longrightarrow_n \lambda y.y \quad \beta$$

- ▶ computes to a value in 1 step using a call-by-name strategy
- ▶ does not compute to a value using a call-by-value strategy

Properties

Values

- ▶ If $M \longrightarrow_v^* V$ and V is a value then $M \Longrightarrow_v V$
- ▶ If $M \longrightarrow_n^* V$ and V is a value then $M \Longrightarrow_n V$

Deterministic

- ▶ If $M \longrightarrow_v N$ and $M \longrightarrow_v P$ then $N = P$
- ▶ If $M \longrightarrow_n N$ and $M \longrightarrow_n P$ then $N = P$

β -equality

- ▶ If $M \longrightarrow_v N$ then $M =_\beta N$
- ▶ If $M \longrightarrow_n N$ then $M =_\beta N$

Evaluation contexts

Although the untyped pure λ -calculus' small-step semantics has only

- ▶ 2 context rules in the call-by-value version
- ▶ 1 context rule in the call-by-name version

real-world programming languages **can have many more**.

Evaluation context provide a way to **compactly** express context rules.

An evaluation context is a term with a special variable $\bullet$, called the **hole**.

If C is an evaluation context, then $C[M]$ represents C with M substituted for $\bullet$.

Every evaluation context C represents a context rule

$$\frac{M \longrightarrow_v N}{C[M] \longrightarrow_v C[N]}$$

expressing that M can be reduced to N in the context C

Evaluation contexts

Values (λ -abstractions of the form $\lambda x.M$) are denoted by V

Call-by-value evaluation contexts

$$C ::= \bullet \mid C M \mid V C$$

Call-by-name evaluation contexts

$$C ::= \bullet \mid C M$$

Note: we use the same context notation as it will be clear from the context whether we mean call-by-value or call-by-name contexts.

Evaluation contexts

Using evaluation contexts, the small-step semantics of the untyped pure λ -calculus can be rephrased as follows:

Call-by-value

$$\frac{}{(\lambda x.M) V \longrightarrow_v M[x \backslash V]} \beta$$

$$\frac{M \longrightarrow_v N}{C[M] \longrightarrow_v C[N]} \text{APP}_C$$

Call-by-name

$$\frac{}{(\lambda x.M) N \longrightarrow_n M[x \backslash N]} \beta$$

$$\frac{M \longrightarrow_n N}{C[M] \longrightarrow_n C[N]} \text{APP}_C$$

Evaluation contexts

The β rule is the only rule where something “interesting” happens, while the context rule shows where it happens in the term.

Note: the evaluation context is mentioned in the rule name to indicate where the reduction is done.

Derivations can always be of the form:

$$\frac{\overline{(\lambda x.M)N \longrightarrow_v M[x \backslash N]}^{\beta}}{C[(\lambda x.M)N] \longrightarrow_v C[M[x \backslash N]]} \text{APP}_C$$

Note: in general other rules and derivations are possible

Evaluation contexts

Booleans:

$$\begin{array}{llll} \mathbf{T} & = & \lambda x. \lambda y. x & \text{and} & = & \lambda a. \lambda b. a \ b \ \mathbf{F} \\ \mathbf{F} & = & \lambda x. \lambda y. y & \text{or} & = & \lambda a. \lambda b. a \ \mathbf{T} \ b \\ & & & \text{not} & = & \lambda a. a \ \mathbf{F} \ \mathbf{T} \end{array}$$

What does $(\text{or}(\text{or } \mathbf{T} \ \mathbf{F})\mathbf{F})$ compute to according to the call-by-value semantics, using evaluation contexts?

$$\frac{\overline{\text{or } \mathbf{T} \longrightarrow_v \lambda b. \mathbf{T} \ \mathbf{T} \ b} \quad \beta}{\text{or}(\text{or } \mathbf{T} \ \mathbf{F})\mathbf{F} \longrightarrow_v \text{or}((\lambda b. \mathbf{T} \ \mathbf{T} \ b)\mathbf{F})\mathbf{F}} \quad \text{APP}_{\text{or}(\bullet \ \mathbf{F})\mathbf{F}}$$

We can derive: $\text{or}(\text{or } \mathbf{T} \ \mathbf{F})\mathbf{F} \longrightarrow_v^* \mathbf{T}$

The first step would require 3 APP rules without evaluation contexts

Normal forms

A **normal form** (w.r.t. a transition system) is one that cannot be reduced further.

A term that has a normal form is said to **converge**, otherwise it is said to **diverge**.

A normal form that is not a value is said to be **stuck**.

Normal forms of the above small-step transition systems:

$$NF := \lambda x.M \mid x\ M \cdots M$$

i.e., **values** or **stuck terms**.

Normal forms

Different strategies might lead to different convergence behaviors and different normal forms:

- ▶ $(\lambda y. \lambda x. x)(\Delta \Delta)$
 - ▶ call-by-value: diverges
 - ▶ call-by-name: converges to $\lambda x. x$
- ▶ $(\lambda x. \lambda y. x)((\lambda z. z)(\lambda w. w))$
 - ▶ call-by-value: converges to $\lambda y. \lambda w. w$
 - ▶ call-by-name: converges to $\lambda y. (\lambda z. z)(\lambda w. w)$
- ▶ $\lambda y. (\lambda z. z)(\lambda w. w)$
 - ▶ call-by-value: is already a value
 - ▶ call-by-name: is already a value

Theorem: If a term converges using call-by-value then it converges using call-by-name.

Normal forms

Not all strategies reduce all β -redexes in normal forms.

- ▶ We saw examples above with redexes left under λ s.
- ▶ A β -normal form is a λ -term without β -redexes.
- ▶ Is there a strategy where normal forms are all β -normal forms?
- ▶ Yes: the normal order reduction.
- ▶ It reduces leftmost outermost β -redexes first.

Normal order reduction

- ▶ reduces under λ s
- ▶ not used in practice

Theorem: If M is β -equal to a β -normal form N , then the normal order reduction reduces M to N

Theorem: call-by-name converges whenever normal order converges

Normal order reduction

Defined as follows (big-step):

$$\overline{x \Longrightarrow_{no} x}$$

$$\frac{M \Longrightarrow_{no} N}{\lambda x.M \Longrightarrow_{no} \lambda x.N}$$

$$\frac{M \Longrightarrow_n \lambda x.P \quad P[x \setminus N] \Longrightarrow_{no} Q}{M N \Longrightarrow_{no} Q}$$

$$\frac{M \Longrightarrow_n P \neq \lambda x.T \quad M \Longrightarrow_{no} Q \quad N \Longrightarrow_{no} R}{M N \Longrightarrow_{no} Q R}$$

Example:

- ▶ $(\lambda x.\lambda y.y)((\lambda x.xx)(\lambda x.xx)) \Longrightarrow_{no} \lambda y.y$
- ▶ but $(\lambda x.\lambda y.x)((\lambda x.xx)(\lambda x.xx))$ does not reduce to a normal form

Conclusion

What did we cover today?

- ▶ small-step operational semantics
- ▶ evaluation contexts
- ▶ normal forms

Next time?

- ▶ The simply-typed λ -calculus